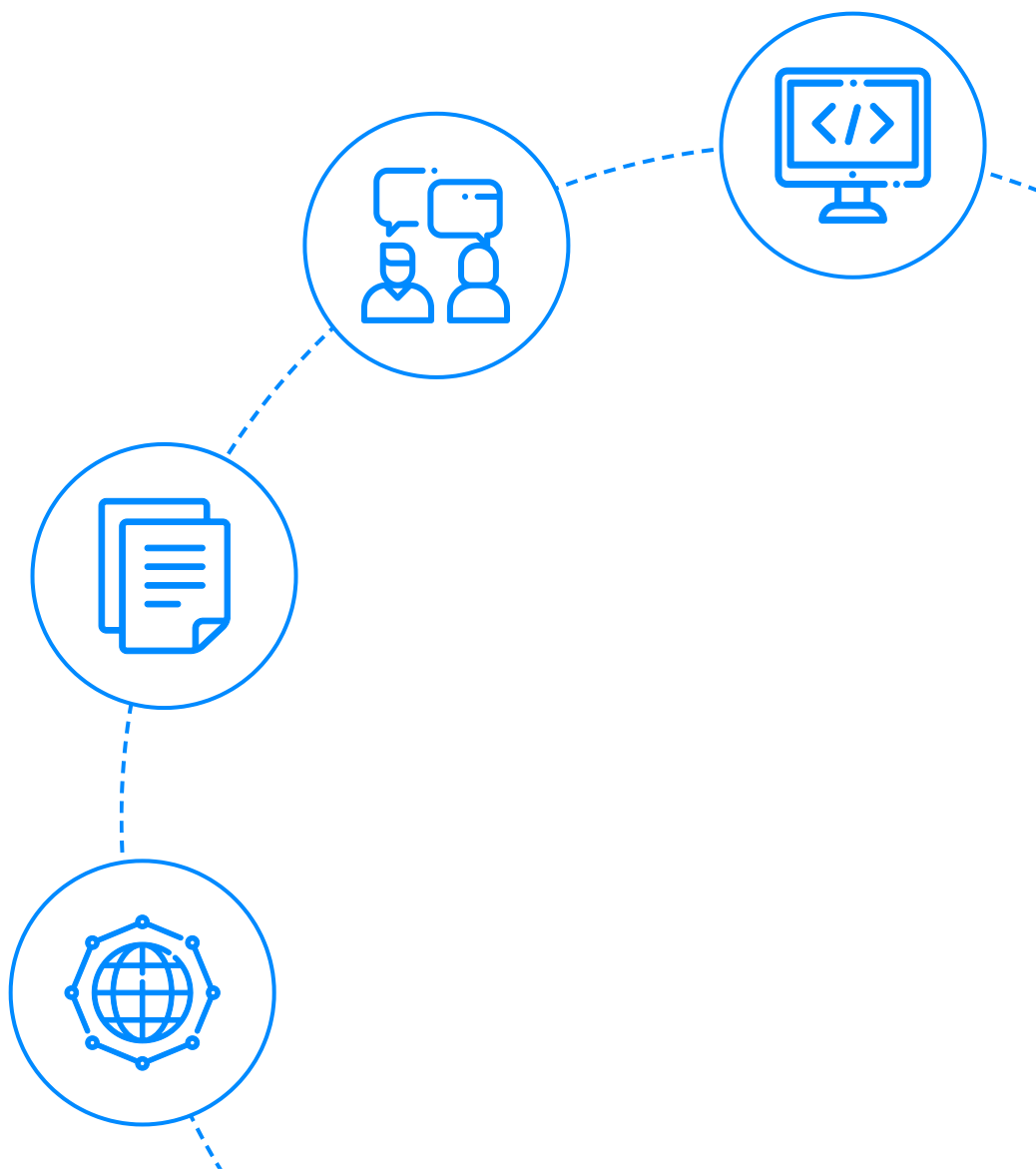




# FastAPI Interview Questions and Answers for Python Developers (2026)



To view the live version of the page, [click here](#).

© Copyright by Interviewbit

# Contents

---

## FastAPI Advanced Interview Questions

1. Explain how to implement custom request validation and serialization beyond Pydantic defaults.
2. How do you design FastAPI microservices and handle inter-service communication?
3. What strategies do you use for caching in FastAPI applications?
4. How do you deploy a FastAPI application with Docker and Uvicorn/Gunicorn?
5. Explain lifespan events (startup/shutdown) in FastAPI and their use cases.
6. How do you implement rate limiting and request throttling in FastAPI?

## FastAPI Common Interview Questions

7. What is middleware in FastAPI and how do you create custom middleware?
8. How do you write tests for FastAPI applications using TestClient?
9. How do you implement CORS (Cross-Origin Resource Sharing) in FastAPI?
10. How do you connect FastAPI with a database using SQLAlchemy?
11. How do you handle errors and exceptions in FastAPI? Explain HTTPException.
12. What is Dependency Injection in FastAPI, and how does it work?

## FastAPI Interview Questions for Experienced

13. How do you handle database migrations in FastAPI using Alembic?
14. Explain APIRouter and how to structure a large FastAPI application.
15. How do you implement WebSockets in FastAPI for real-time communication?
16. What are Background Tasks in FastAPI, and when should you use them?
17. How do you implement OAuth2 with JWT authentication in FastAPI?

## FastAPI Interview Questions for Experienced

(.....Continued)

18. Explain async and await in FastAPI. When should you use async def vs def?

## FastAPI Interview Questions for Freshers

19. What is the purpose of response\_model in FastAPI path operations?
20. How do you handle form data and file uploads in FastAPI?
21. Explain the different HTTP methods (GET, POST, PUT, DELETE, PATCH) and their usage in FastAPI.
22. How do you define request body using Pydantic models?
23. What are path parameters and query parameters in FastAPI? How do you define them?
24. How do you create a basic FastAPI application with a GET endpoint?

## Python FastAPI Interview Questions (Fundamentals)

25. What is FastAPI, and what are its key features?
26. Compare FastAPI with Flask and Django REST Framework.
27. How does FastAPI automatically generate OpenAPI (Swagger) documentation?
28. What is Pydantic, and why is it integral to FastAPI?
29. What is Starlette, and how does FastAPI build upon it?
30. Explain the difference between ASGI and WSGI. Why does FastAPI use ASGI?

# Let's get Started

---

A 2026-ready FastAPI interview guide with questions and answers on async/await, Pydantic models, dependency injection, path operations, OAuth2 authentication, WebSockets, background tasks, middleware, testing, and deployment for freshers & experienced Python developers. (InterviewBit)

## FastAPI Advanced Interview Questions

### 1. Explain how to implement custom request validation and serialization beyond Pydantic defaults.

Pydantic handles most validation tasks, but advanced applications may require custom validation logic. FastAPI allows developers to extend validation using custom validators and serialization rules.

Example:

```
from pydantic import BaseModel, validator
```

```
class User(BaseModel): username: str password: str
```

```
@validator("password") def password_length(cls, value): if len(value) < 8: raise  
ValueError("Password must be at least 8 characters long") return value
```

Developers can also create custom response serialization by modifying output data before returning it from endpoints.

Custom validation is useful when:

- Business rules require complex validation logic
- Input data must be transformed before processing
- Security checks must be enforced

### 2. How do you design FastAPI microservices and handle inter-service communication?

FastAPI is commonly used to build microservices where each service handles a specific business function. Microservice architecture separates applications into independent services that communicate through APIs or messaging systems.

**Design considerations include:**

- Independent service deployment
- Separate databases for each service
- Clear API contracts between services
- Centralized authentication and logging

**Communication methods include:**

**REST APIs**

- Services communicate using HTTP endpoints.

**Message Queues**

- Tools like RabbitMQ or Kafka enable asynchronous communication.

**Service Discovery**

- Allows services to locate each other dynamically.

Microservices improve scalability and allow teams to develop and deploy services independently.

### 3. What strategies do you use for caching in FastAPI applications?

Caching reduces repeated database or external API calls by storing frequently requested data. This improves performance and reduces server load.

Common caching strategies include:

**In-Memory Caching**

- Stores data in application memory
- Suitable for small datasets
- Example: Python dictionaries or cache libraries

**Redis Caching**

- Stores cached data in a separate service
- Supports distributed applications
- Commonly used in production environments

### Response Caching

- Stores full API responses
- Reduces processing time for repeated requests

Example using Redis-based caching:

```
import aioredis
```

```
redis = await aioredis.from_url("redis://localhost")
```

```
async def get_cached_data(key): return await redis.get(key)
```

Caching improves application speed and reduces infrastructure cost.

## 4. How do you deploy a FastAPI application with Docker and Uvicorn/Gunicorn?

FastAPI applications are typically deployed using ASGI servers such as Uvicorn or Gunicorn combined with Uvicorn workers. Docker is used to containerize the application and ensure consistent deployment.

Example Dockerfile:

```
FROM python:3.10
```

```
WORKDIR /app
```

```
COPY. /app
```

```
RUN pip install -r requirements.txt
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

For production environments, Gunicorn is often used with Uvicorn workers:

```
gunicorn main:app -k uvicorn.workers.UvicornWorker
```

**Deployment benefits include:**

- Consistent environment setup
- Easy scaling using containers
- Simplified CI/CD integration

## 5. Explain lifespan events (startup/shutdown) in FastAPI and their use cases.

Lifespan events allow developers to execute code when the application starts or stops. They are useful for managing resources that must be initialized or cleaned up during application lifecycle.

Example:

```
from fastapi import FastAPI
app = FastAPI()
@app.on_event("startup")
async def startup_event(): print("Application started")
@app.on_event("shutdown")
async def shutdown_event(): print("Application stopped")
```

### Common use cases include:

- Opening database connections
- Loading machine learning models
- Initializing cache or message queues
- Closing resources during shutdown

Lifespan events help manage application resources efficiently.

## 6. How do you implement rate limiting and request throttling in FastAPI?

Rate limiting controls how many requests a client can send within a specific time period. It prevents server overload, protects APIs from abuse, and improves service stability.

FastAPI does not include built-in rate limiting, but it can be implemented using middleware or third-party libraries such as **slowapi**.

Example using SlowAPI:

```
from slowapi import Limiter
from slowapi.util import get_remote_address
limiter = Limiter(key_func=get_remote_address)
@app.get("/items")
@limiter.limit("5/minute")
async def get_items(): return {"message": "Rate limited endpoint"}
```

**Common rate-limiting strategies include:**

- Limiting requests per IP address
- Limiting requests per authenticated user
- Using Redis or caching systems to track request counts

Rate limiting helps maintain API reliability and protects backend resources.

## FastAPI Common Interview Questions

### 7. What is middleware in FastAPI and how do you create custom middleware?

Middleware is code that runs before or after every request. It is used for logging, authentication checks, request tracking, and response modification.

Custom middleware can be created using the **@app.middleware** decorator.

Example:

```
from fastapi import Request
import time
@app.middleware("http")
```



```
async def log_request_time(request: Request, call_next): start_time = time.time()
response = await call_next(request) process_time = time.time() - start_time
response.headers["X-Process-Time"] = str(process_time) return response
```

**Explanation:**

- The middleware runs before and after each request.
- **call\_next()** forwards the request to the endpoint.
- The middleware modifies the response headers.

Middleware is useful for monitoring and cross-cutting application logic.

## 8. How do you write tests for FastAPI applications using TestClient?

FastAPI provides a **TestClient** class that allows developers to test endpoints without running the server.

Example:

```
from fastapi.testclient import TestClient
```

```
client = TestClient(app)
```

```
def test_read_root(): response = client.get("/") assert response.status_code == 200
assert response.json() == {"message": "Hello World"}
```

**Explanation:**

- **TestClient** simulates HTTP requests.
- Tests verify response status codes and output data.
- It helps developers validate API behavior during development.

Testing ensures APIs behave correctly and prevents bugs in production deployments.

## 9. How do you implement CORS (Cross-Origin Resource Sharing) in FastAPI?

CORS allows a frontend application running on one domain to access an API hosted on another domain. Without CORS configuration, browsers block cross-domain requests.

FastAPI supports CORS using **CORSMiddleware**.

Example:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
allow_methods=["*"], allow_headers=["*"], )
```

**Explanation:**

- **allow\_origins** defines which domains can access the API.
- **allow\_methods** controls allowed HTTP methods.
- **allow\_headers** allows specific headers.

CORS configuration is important when connecting frontend applications with backend APIs.

## 10. How do you connect FastAPI with a database using SQLAlchemy?

SQLAlchemy is commonly used as an ORM (Object Relational Mapper) to interact with databases in FastAPI applications.

Basic steps include:

1. Create a database engine and session.
2. Define database models.
3. Use dependency injection to manage database sessions.
4. Perform CRUD operations inside route functions.

Example:

```
from sqlalchemy.orm import Session

from fastapi import Depends

def get_db(): db = SessionLocal() try: yield db finally: db.close()

@app.get("/users/")

def get_users(db: Session = Depends(get_db)): return db.query(User).all()
```

**Explanation:**

- **get\_db()** manages the database session lifecycle.
- The session is injected using **Depends**.
- The ORM handles database queries using Python objects.

## 11. How do you handle errors and exceptions in FastAPI? Explain HTTPException.

FastAPI provides built-in tools to handle runtime errors and validation failures. The most commonly used exception class is `HTTPException`.

Example:

```
from fastapi import HTTPException

@app.get("/items/{item_id}")
def read_item(item_id: int):
    if item_id != 1:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id}
```

**Explanation:**

- **status\_code** defines the HTTP response code.
- **The detail** provides an error message returned to the client.

FastAPI automatically converts exceptions into structured JSON responses. Developers can also create global exception handlers for application-wide error handling.

## 12. What is Dependency Injection in FastAPI, and how does it work?

Dependency Injection allows developers to reuse logic such as authentication, database connections, and configuration across multiple endpoints. Instead of writing the same setup code in every route, FastAPI allows dependencies to be defined separately and injected when required.

FastAPI uses the **Depends** function to manage dependencies.

Example:

```
from fastapi import Depends, FastAPI
app = FastAPI()
def common_parameters(): return {"limit": 10}
@app.get("/items/")
def read_items(params: dict = Depends(common_parameters)): return params
```

**Explanation:**

- The **common\_parameters** function acts as a dependency.
- **Depends()** tells FastAPI to execute the dependency before running the route.
- The returned value is passed to the endpoint.

Dependency injection improves code reuse and simplifies application maintenance.

## FastAPI Interview Questions for Experienced

### 13. How do you handle database migrations in FastAPI using Alembic?

Alembic is a database migration tool used with SQLAlchemy to manage schema changes. It allows developers to update the database structure without losing existing data.

Migration process involves:

1. Install and initialize Alembic.
2. Configure database connection.
3. Generate migration scripts.
4. Apply migration to the database.

Common commands:

```
alembic init alembic
```

```
alembic revision --autogenerate -m "Create tables"
```

```
alembic upgrade head
```

Alembic tracks schema versions and ensures database changes remain consistent across development and production environments.

## 14. Explain APIRouter and how to structure a large FastAPI application.

**APIRouter** helps organize routes into separate modules. It is used when applications grow large and contain multiple endpoints.

Example:

```
from fastapi import APIRouter
router = APIRouter()
@router.get("/items")
def get_items(): return {"items": []}
```

Routes are then included in the main application:

```
app.include_router(router)
```

Using APIRouter helps:

- Separate functionality into modules
- Improve code readability
- Support team-based development
- Simplify maintenance

Large FastAPI applications are usually structured with folders for routers, models, database logic, and services.

## 15. How do you implement WebSockets in FastAPI for real-time communication?

WebSockets allow continuous two-way communication between client and server. They are used in chat applications, live notifications, and streaming dashboards.

Example:

```
from fastapi import WebSocket
```

```
@app.websocket("/ws")
```

```
async def websocket_endpoint(websocket: WebSocket): await websocket.accept()
while True: message = await websocket.receive_text() await
websocket.send_text(f"Message received: {message}")
```

WebSockets maintain an open connection instead of sending separate HTTP requests. This allows real-time data exchange.

## 16. What are Background Tasks in FastAPI, and when should you use them?

Background tasks allow certain operations to run after sending a response to the client. This prevents delays in API response time.

They are useful for:

- Sending emails
- Logging events
- Processing uploaded files
- Generating reports

Example:

```
from fastapi import BackgroundTasks
```

```
def write_log(message: str): with open("log.txt", "a") as file: file.write(message)
```

```
@app.post("/notify/")
```

```
def send_notification(background_tasks: BackgroundTasks):
```

```
background_tasks.add_task(write_log, "Notification sent") return {"message": "Task
scheduled"}
```

Background tasks improve user experience by handling time-consuming operations separately.

## 17. How do you implement OAuth2 with JWT authentication in FastAPI?

OAuth2 with JWT is commonly used to secure APIs by verifying user identity and managing access permissions.

The authentication process involves:

1. User submits login credentials.
2. Server validates credentials.
3. Server generates a JWT token.
4. Client sends the token with future requests.
5. Server verifies the token before allowing access.

Example:

```
from fastapi.security import OAuth2PasswordBearer
from fastapi import Depends
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
@app.get("/users/me")
async def read_users_me(token: str = Depends(oauth2_scheme)): return {"token": token}
```

In production applications:

- JWT tokens store encoded user data.
- Tokens are validated using secret keys.
- Expiration time prevents unauthorized long-term access.

OAuth2 with JWT provides stateless authentication and improves API security.

## 18. Explain `async` and `await` in FastAPI. When should you use `async def` vs `def`?

FastAPI supports asynchronous programming using **`async`** and **`await`**. These keywords allow the server to handle multiple requests without blocking execution.

**`async def`**

- Used when route functions perform operations that involve waiting.
- Suitable for database calls, API calls, or file operations.
- Allows FastAPI to process other requests while waiting for a response.

Example:

```
@app.get("/users/")
```

```
async def get_users(): users = await fetch_users_from_service() return users
```

**def**

- Used for synchronous operations.
- Suitable when functions execute quickly and do not require waiting for external resources.

Example:

```
@app.get("/status/")
```

```
def check_status(): return {"status": "Running"}
```

**When to use which:**

- Use **async def** when performing network requests, database operations, or long-running tasks.
- Use **def** for simple calculations or immediate responses.

Using async functions improves performance in high-traffic applications.

## FastAPI Interview Questions for Freshers

### 19. What is the purpose of `response_model` in FastAPI path operations?

The **response\_model** defines the structure of the data returned by an API. It ensures that responses follow a specific format and hides unnecessary fields.

Example:

```
class ItemResponse(BaseModel): name: str price: float
```

```
@app.get("/items/{item_id}", response_model=ItemResponse)
```



```
def get_item(item_id: int): return {"name": "Laptop", "price": 50000,  
    "internal_code": "XYZ"}
```

In this case, the response will only include fields defined in **ItemResponse**. The **internal\_code** field is excluded.

The **response\_model** improves data consistency, security, and API documentation accuracy.

## 20. How do you handle form data and file uploads in FastAPI?

FastAPI supports form data and file uploads using special classes.

### Handling Form Data

```
from fastapi import Form  
  
@app.post("/login/")  
def login(username: str = Form(...), password: str = Form(...)): return {"username":  
    username}
```

**Form(...)** indicates that the input is received as form data.

### Handling File Uploads

```
from fastapi import UploadFile, File  
  
@app.post("/upload/")  
def upload_file(file: UploadFile = File(...)): return {"filename": file.filename}
```

**UploadFile** allows efficient file handling and supports streaming large files.

## 21. Explain the different HTTP methods (GET, POST, PUT, DELETE, PATCH) and their usage in FastAPI.

HTTP methods define the type of operation performed on resources.

### GET:

- Retrieves data from the server.
- Used to fetch records or resources.

### POST:

- Creates new data.
- Used when adding new entries or records.

**PUT:**

- Updates an entire resource.
- Used when replacing existing data completely.

**PATCH:**

- Updates part of a resource.
- Used when modifying specific fields.

**DELETE:**

- Removes a resource.
- Used when deleting records.

FastAPI provides decorators such as **@app.get()**, **@app.post()**, and **@app.put()** to define these operations.

## 22. How do you define request body using Pydantic models?

FastAPI uses Pydantic models to define the request body structure. Pydantic ensures that input data follows the required format.

Example:

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool

@app.post("/items/")
def create_item(item: Item):
    return item
```

**Explanation:**

- **BaseModel** is used to create a request model.
- FastAPI validates request data automatically.
- The input is converted into a Python object.

If the client sends incorrect data, FastAPI returns a validation error.

## 23. What are path parameters and query parameters in FastAPI? How do you define them?

FastAPI uses parameters to receive data from client requests.

### Path Parameters

Path parameters are part of the URL path. They identify specific resources.

Example:

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int):
```

```
    return {"item_id": item_id}
```

Here, **item\_id** is extracted from the URL and validated as an integer.

### Query Parameters

Query parameters are added after the **?** symbol in the URL and are used for filtering or optional input.

Example:

```
@app.get("/items/")
```

```
def read_items(limit: int = 10):
```

```
    return {"limit": limit}
```

If the client does not provide a value, the default value is used.

Path parameters identify resources, while query parameters modify or filter responses.

## 24. How do you create a basic FastAPI application with a GET endpoint?

A basic FastAPI application starts by importing the FastAPI class, creating an application instance, and defining route functions.

Example:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Hello World"}
```

**Explanation:**

- FastAPI() creates the application object.
- @app.get("/") defines a GET endpoint.
- The function returns a JSON response.

This structure creates a simple API endpoint that returns data when a client sends a GET request.

## Python FastAPI Interview Questions (Fundamentals)

### 25. What is FastAPI, and what are its key features?

FastAPI is a Python web framework used to build APIs. It allows developers to create endpoints that accept client requests, process data, and return structured responses. The framework focuses on request validation, performance, and automatic documentation.

Key features of FastAPI include:

**Automatic Data Validation**

FastAPI uses Python type hints to validate request data. When users send input to an API, FastAPI checks whether the data matches the expected format before executing business logic.

**Asynchronous Request Handling**

FastAPI supports async functions, which allow the server to process multiple requests at the same time without blocking execution.

## Automatic API Documentation

FastAPI generates OpenAPI and Swagger documentation automatically when routes and models are defined.

## Dependency Injection Support

It allows reusable logic such as authentication, database connections, and configuration handling.

## Structured Error Handling

FastAPI returns clear validation and error responses, which help developers debug issues quickly.

## 26. Compare FastAPI with Flask and Django REST Framework.

FastAPI, Flask, and Django REST Framework are all used for building APIs, but they differ in design and use cases.

### FastAPI

- Designed specifically for API development
- Supports asynchronous programming
- Provides automatic validation and documentation
- Commonly used for microservices and real-time applications

### Flask

- Minimal and flexible framework
- Requires external libraries for validation and documentation
- Suitable for small applications and custom backend services

### Django REST Framework

- Built on Django's full-stack architecture
- Includes ORM, authentication system, and admin interface
- Suitable for large applications requiring database management and user administration

FastAPI is generally chosen when performance, async support, and quick API development are priorities.

## 27. How does FastAPI automatically generate OpenAPI (Swagger) documentation?

FastAPI generates API documentation by analyzing route definitions, request models, and response schemas. When developers define input and output models using type annotations and Pydantic classes, FastAPI uses this information to create OpenAPI specifications.

The framework provides two documentation interfaces:

### Swagger UI

An interactive interface that allows developers to test API endpoints directly from the browser.

### ReDoc Interface

A structured documentation view used for reading API details.

The documentation is automatically updated when developers modify routes or data models. This reduces manual documentation effort and improves API testing and collaboration.

## 28. What is Pydantic, and why is it integral to FastAPI?

Pydantic is a Python library used for data validation and parsing. It allows developers to define structured data models using Python classes and type annotations.

In FastAPI, Pydantic is used to:

- Validate incoming request data
- Convert request data into Python objects
- Validate response models
- Generate API documentation schemas

For example, when a developer defines a request model using Pydantic, FastAPI automatically checks whether the input matches the required fields and data types. If the validation fails, FastAPI returns an error response without executing the route logic.

Pydantic ensures data consistency and reduces manual validation code in FastAPI applications.

## 29. What is Starlette, and how does FastAPI build upon it?

Starlette is a lightweight ASGI framework used for building asynchronous web services. It provides core web features such as routing, middleware support, request handling, and WebSocket communication.

FastAPI uses Starlette as its foundation. Starlette handles:

- HTTP request routing
- Middleware execution
- Background task management
- WebSocket support

FastAPI builds on Starlette by adding higher-level features such as data validation, dependency injection, and automatic API documentation. This allows developers to use Starlette's performance while working with structured API development tools.

## 30. Explain the difference between ASGI and WSGI. Why does FastAPI use ASGI?

ASGI and WSGI are interfaces that allow web servers to communicate with Python applications.

### WSGI (Web Server Gateway Interface)

WSGI supports synchronous request handling. Each request is processed one at a time, which can slow down applications handling many simultaneous requests. Frameworks like Flask and Django traditionally use WSGI.

### ASGI (Asynchronous Server Gateway Interface)

ASGI supports asynchronous communication. It allows servers to handle multiple requests concurrently using event-driven execution. ASGI also supports features such as WebSockets and long-running connections.

FastAPI uses ASGI because it allows the framework to handle high concurrency and real-time applications efficiently. This improves performance in systems that receive frequent or simultaneous client requests.





# Links to More Interview Questions

---

[Accenture Interview Questions](#)

[Accolite Interview Questions](#)

[Adobe Interview Questions](#)

[Amazon Interview Questions](#)

[Amdocs Interview Questions](#)

[Apisero Interview Questions](#)

[Arcesium Interview Questions](#)

[Avasoft Interview Questions](#)

[Axis Bank Interview Questions](#)

[Capgemini Interview Questions](#)

[Cisco Interview Questions](#)

[Cognizant Interview Questions](#)

[Dassault Systemes Interview Questions](#)

[De Shaw Interview Questions](#)

[Delhivery Interview Questions](#)

[Deloitte Interview Questions](#)

[Directi Interview Questions](#)

[Dxc Interview Questions](#)

[Epam Interview Questions](#)

[Ey Interview Questions](#)

[Facebook Interview Questions](#)

[Flipkart Interview Questions](#)

[Genc Interview Questions](#)

[Genpact Interview Questions](#)

[Goldman Sachs Interview Questions](#)

[Google Interview Questions](#)

[Hashedin Interview Questions](#)

[Hcl Interview Questions](#)

[Hexaware Interview Questions](#)

[Ibm Interview Questions](#)

[Infosys Interview Questions](#)

[Intuit Interview Questions](#)

[Jp Morgan Interview Questions](#)